Listing 9.12 HTML for our dashboard with a brush

```
<!doctype html>
<html>
<script src="d3.v3.min.js" type="text/JavaScript"></script>
<body onload="dashboard()" >
<svg id="leftSVG" class="svgDash"s></svg>
<svg id="rightSVG"  class="svgDash"></svg>
<div id="brush"></div>
<div id="spreadsheet"></div>
</body>
</html>
```

This requires that we add a reference in the CSS to our new div and slightly adjust the `#spreadsheet` div to take up less room.

Listing 9.13 Brush div CSS changes

```
#brush {
    overflow: auto;
    width: 100%;
    height: 20%;
}
#spreadsheet {
    overflow: auto;
    width: 100%;
    height: 30%;
}
```

Because we need a scale to build a brush, we'll start by making a new scale that suits our tweet data: `d3.time.scale`. This scale is designed to take JavaScript Date datatypes and deal with them like the more common `d3.scale.linear`. To populate the domain of our new scale, we need to find the earliest and latest times of each tweet. We do this by using `d3.extent` and built-in JavaScript array mapping functionality to create an array of `Date` datatypes from the string representation of the times of each tweet.

A brush is an interactive collection of components that allows a user to drag one end of the brush to designate an extent, or to move that extent to a different range. Typical brush aspects are explained in figure 9.8. In this chapter we only create a brush that allows selection along the x-axis, but if you want to see a brush that selects along the x- and y-axes, you can check out chapter 11, where we use it to select points laid out on an xy plane.

It's also helpful to create an axis to go along with our brush. The brush is created as a region of interactivity, and clicking on that region produces a rectangle in response. But before any interaction, the area looks blank. By including an axis, we inform the user of the range attached to this brush. Our new axis needs more particular formatting because we're using a time scale.

After that, we'll create a brush and assign the time scale to the brush's `.x()` function. We can also create brushes that are vertical or allow for selecting a region by setting the `.y()` function. We'll assign an event listener that listens for the custom